

Informe de arquitectura — Proyecto Maxes

Web / Maxes Gestor

1. Objetivo general

El objetivo del proyecto es reemplazar y modernizar la operatoria actual de catálogo web y recepción de pedidos, separando claramente la parte pública de la parte administrativa.

Se propone dividir la solución en dos proyectos independientes:

- **maxes_web**: sitio público para clientes.
- **maxes_gestor**: sistema interno de administración, operación y control.

La premisa principal es que **maxes_gestor será el cerebro del sistema**, mientras que **maxes_web será únicamente el canal público de catálogo y recepción de pedidos**.

2. Arquitectura general

```
maxes_gestor
Base interna principal
Fuente de verdad del negocio

      ↓ publicación / sincronización

maxes_web
Base pública independiente
Catálogo publicado + pedidos web
```

La base de datos de la web será independiente de la base del gestor. La web no administrará artículos, precios, stock ni clientes reales. Solo mostrará información previamente publicada desde el gestor y permitirá recibir pedidos.

3. Proyecto maxes_gestor

Rol del proyecto

`maxes_gestor` será el sistema principal de administración. Desde allí se gestionarán los datos reales del negocio y se decidirá qué información se publica en la web.

Tecnología propuesta

- Frontend: **React + TypeScript + Refine**
- Backend: **Node.js**, preferentemente **NestJS** o Express
- Base de datos: **SQL Server**

- Acceso: interno, red privada o acceso protegido
- Usuarios: con roles y permisos

Base de datos

Se propone mantener **SQL Server** para el gestor, ya que es la base de datos que actualmente se utiliza y es adecuada para la operatoria administrativa/comercial.

Tablas principales esperadas:

```
articulos
rubros
clientes
proveedores
transportes
vendedores
precios
stock
pedidos
pedidos_items
usuarios
roles
parametros
publicaciones_web
```

Secciones principales

```
Dashboard
Pedidos web
Artículos
Rubros
Clientes
Precios / listas
Stock
Publicación de catálogo
Vendedores
Proveedores
Transportes
Comprobantes
Cuentas corrientes
Caja
Reportes
RMA
Parámetros
Usuarios y permisos
Actividad / auditoría
```

Funcionalidades clave

- Administrar la base real de artículos.
 - Administrar rubros/categorías.
 - Administrar clientes.
 - Administrar precios.
 - Controlar stock.
 - Revisar pedidos recibidos desde la web.
 - Confirmar, modificar o rechazar pedidos.
 - Publicar artículos hacia la web.
 - Bloquear o habilitar el catálogo público.
 - Ver alertas y actividad en tiempo real.
 - Auditar publicaciones y cambios.
-

4. Proyecto maxes_web

Rol del proyecto

`maxes_web` será el sitio público utilizado por los clientes. Su responsabilidad será mostrar el catálogo publicado y recibir pedidos.

No será un sistema administrativo.

Tecnología propuesta

- Frontend: **Next.js + React + TypeScript**
- Backend/API: API liviana en Next.js o Node.js separado
- Base de datos: independiente de la base interna
- Hosting: Vercel, Render, Railway, VPS o infraestructura Node equivalente

Base de datos de la web

La base de datos de `maxes_web` debe ser independiente y abastecida desde `maxes_gestor`.

Opciones:

1. SQL Server también para la web

2. Ventaja: misma tecnología que el gestor.

3. Desventaja: puede complicar hosting y costos si se busca una infraestructura web más liviana.

4. PostgreSQL para la web

5. Ventaja: muy buena opción para hosting moderno, económica y flexible.

6. Desventaja: requiere manejar dos motores distintos.

7. Catálogo cacheado / JSON publicado

8. Ventaja: extremadamente rápido para lectura.

9. Desventaja: menos flexible si luego se necesitan consultas complejas.

Recomendación

Para `maxes_gestor`, mantener **SQL Server**.

Para `maxes_web`, evaluar **PostgreSQL** como opción recomendada si se busca una infraestructura moderna, económica y simple de publicar. Si se prioriza mantener una sola tecnología de base de datos, puede usarse SQL Server también para la web.

Tablas principales de maxes_web

```
rubros_web
articulos_web
imagenes_articulos_web
pedidos_web
pedidos_web_items
clientes_web
configuracion_web
logs_email
```

Secciones públicas

```
Home / catálogo
Rubros / categorías
Buscador de artículos
Listado de artículos
Ficha rápida de artículo
Carrito / pedido
Datos del cliente
Confirmación de pedido
Catálogo bloqueado / mantenimiento
```

Funcionalidades clave

- Mostrar catálogo público.
- Filtrar por rubro/categoría.
- Buscar artículos.
- Mostrar código, descripción, precio e imagen.
- Armar pedido.
- Solicitar datos mínimos del cliente.
- Guardar pedido web.
- Enviar confirmación por email al cliente.
- Mostrar número de pedido.
- Permitir bloqueo temporal del catálogo si el gestor lo indica.

5. Publicación de artículos

Los artículos nacen y se administran en `maxes_gestor`.

La web no crea ni modifica artículos.

Flujo propuesto:

```
Gestor administra artículos reales
↓
Usuario marca artículos visibles para web
↓
Gestor ejecuta "Publicar catálogo"
↓
Se actualiza la base de maxes_web
↓
El cliente ve el catálogo actualizado
```

La tabla `articulos` del gestor contendrá toda la información interna del producto:

```
codigo
descripcion
costo
precio
proveedor
stock
stock_minimo
margen
observaciones internas
activo
```

La tabla `articulos_web` solo contendrá los datos públicos:

```
articulo_id_origen
codigo
descripcion_publica
precio_publicado
rubro
imagen
visible
destacado
orden
fecha_publicacion
```

6. Recepción de pedidos

El pedido se genera en la web, pero no debe considerarse venta definitiva hasta que sea revisado en el gestor.

Flujo propuesto:

```
Cliente arma pedido en maxes_web
↓
maxes_web guarda pedido
↓
maxes_web genera número de pedido
↓
maxes_web envía email de confirmación al cliente
↓
maxes_gestor detecta nuevo pedido
↓
Operador revisa el pedido
↓
El pedido se confirma, modifica o rechaza
```

Estados sugeridos:

```
nuevo
visto
en_revision
confirmado
rechazado
cancelado
importado
```

Email al cliente

El envío de email al cliente debe ser obligatorio en el MVP.

El email debe informar:

```
Número de pedido
Fecha y hora
Datos del cliente
Detalle de artículos solicitados
Cantidades
Total estimado, si corresponde
Aclaración de que el pedido queda sujeto a revisión
```

El pedido debe guardarse aunque falle el email.

Campos sugeridos:

```
email_estado  
email_fecha  
email_error  
email_reintentos
```

No se considera obligatorio enviar email interno, porque `maxes_gestor` tendrá alertas y actividad en tiempo real.

7. Separación de responsabilidades

`maxes_gestor`

```
Administra  
Controla  
Valida  
Publica  
Confirma pedidos  
Gestiona operación real
```

`maxes_web`

```
Muestra catálogo  
Recibe pedidos  
Confirma recepción al cliente  
No administra datos comerciales internos
```

8. Justificación de Next.js para maxes_web

Actualmente la empresa trabaja con React, TypeScript y Refine. Esa tecnología sigue siendo adecuada para `maxes_gestor`.

Sin embargo, para `maxes_web` se propone usar **Next.js** porque el objetivo es distinto.

`maxes_web` será una web pública, orientada a clientes, con necesidad de:

```
Carga rápida  
Buen comportamiento en celulares  
Buen SEO  
Catálogo indexable  
Optimización de imágenes
```

Mejor experiencia inicial de carga
Posibilidad de cachear contenido

Una aplicación React tradicional tipo SPA normalmente funciona así:

El navegador descarga JavaScript
Luego ejecuta la aplicación
Luego consulta datos
Luego renderiza el catálogo

En cambio, Next.js permite servir contenido ya preparado o cacheado:

El navegador recibe HTML inicial más completo
El cliente ve contenido antes
Luego se agregan las interacciones de React

Esto es especialmente útil para un catálogo público con muchos productos e imágenes.

Por qué no Refine en la web pública

Refine es excelente para paneles administrativos, ABM, dashboards y CRUD internos.

Pero `maxes_web` no es un administrador. Es una web pública para clientes.

Por eso:

Refine → recomendado para `maxes_gestor`
Next.js → recomendado para `maxes_web`

Beneficios concretos de Next.js

Mejor primera carga
Mejor rendimiento percibido
Mejor soporte para SEO
Optimización automática de imágenes
Rutas públicas más ordenadas
Posibilidad de generar páginas por rubro/producto
Mejor compatibilidad con hosting moderno

9. Infraestructura propuesta

maxes_gestor

Frontend interno:
React + TypeScript + Refine

Backend:
Node.js / NestJS

Base de datos:
SQL Server

Hosting:
Servidor interno, VPS privado o nube protegida

Acceso:
Usuarios internos con autenticación
Roles y permisos

maxes_web

Frontend público:
Next.js + React + TypeScript

API:
Next.js API routes o backend Node separado

Base de datos:
PostgreSQL recomendado
SQL Server posible si se prioriza uniformidad

Hosting:
Vercel, Render, Railway, VPS o servidor Node

Acceso:
Público para clientes
API protegida para publicación desde gestor

10. Conclusión

La solución recomendada es separar claramente el sistema en dos proyectos:

maxes_gestor = cerebro del negocio
maxes_web = canal público de catálogo y pedidos

Cada proyecto tendrá su propia base de datos.

`maxes_gestor` mantendrá la base completa y real del negocio. Desde allí se publicará hacia `maxes_web`.

`maxes_web` tendrá una base pública independiente, con artículos publicados, rubros publicados, imágenes y pedidos web.

Esta arquitectura mejora:

- Seguridad
- Rendimiento
- Mantenibilidad
- Escalabilidad
- Claridad funcional
- Independencia entre web pública y gestión interna

Además, permite modernizar la web sin comprometer la operación administrativa, y permite evolucionar el gestor como sistema central de negocio.